

Type Driven Design

Typing the Untypeable.

Avec DuploJS 

C'est quoi l'intypable ?

Définir qu'une variable est une `string` c'est facile, mais définir qu'une variable est une **adresse email** l'est beaucoup moins !

```
const email: `${string}@${string}.${string}` = "@@superEmail@tru....c.commmmmmmmm";
```

Une **adresse email** n'est pas un type fondamental du langage. Le langage sait représenter du texte avec `string`, mais ce n'est pas son rôle de connaître toutes les règles du monde réel. C'est donc au code de l'application de vérifier qu'une chaîne de caractères est bien une adresse email valide.

L'**intypable**, ce sont les contraintes qui ne peuvent être **vérifiées** qu'à travers l'**exécution de code**.



Quels types existent en TypeScript ?

```
const firstname: string = "Mathieu";
const age: number = 23;
const quantity: bigint = 100n;
const isTeacher: boolean = true;
const uniqueId: symbol = Symbol("id");
const empty: undefined = undefined;
const nil: null = null;
const anything: any = "n'importe quelle valeur";
const unknownValue: unknown = "valeur a verifier avant utilisation";
const noValue: void = undefined;
function fail(message: string): never {
    throw new Error(message);
}
const user: object = {
    firstname: "Mathieu",
    age: 23,
};
const notes: number[] = [12, 15, 18];
const tags: Array<string> = ["type", "typescript"];
const position: [number, number] = [10, 20];
const greet: (name: string) => string = (name) => `Bonjour ${name}`;
```

```
const status: "draft" | "published" = "draft";
const identifier: string | number = "user-1";
const person: { firstname: string } & { age: number } = {
    firstname: "Mathieu",
    age: 23,
};
type User = {
    firstname: string;
    age: number;
};
interface Product {
    name: string;
    price: number;
}
enum Direction {
    Up,
    Down,
    Left,
    Right,
}
```



Quelles possibilités de typage existent en TypeScript ?

```
// Annotation de type
const age: number = 23;

// Annotation type paramètre/retour.
function wrap(value: string): { value: string } {
    return { value };
}

// Inférence
const result = wrap("superValue")

interface MyObject {
    prop1: string,
    values: Record<string, number>
}

const myObject: MyObject = {
    prop1: 12 as any, // Assertion de type
    values: {
        one: 1,
        two: 2,
    } satisfies Record<"one" | "two", unknown> // Annotation secondaire,
}
```

```
class SuperClass implements MyObject { // Implémentation
    prop1: string = "12";
    values: Record<string, number> = {};
}

abstract class OtherClass {
    abstract superMethods(value: number): string
}

class SubClass extends OtherClass { // Abstraction
    superMethods(value: number): string {
        return value.toString();
    }
}

// Predicate
function isString(value: unknown): value is string {
    return typeof value === "string";
}

function assertsNumber(value: unknown): asserts value is string {
    if(typeof value !== "number") {
        throw new Error("is not number");
    }
}
```



Commençons par un exemple simple : une fonction de division

Cette fonction simple prend en entrée le `dividend` et le `divisor`, effectue l'opération, puis retourne le résultat.

```
function division(dividend, divisor) {  
    return dividend / divisor;  
}
```

```
division(10, 5);  
division("toto", 50);
```



Commençons par un exemple simple : une fonction de division

Première contrainte : une division ne peut être faite qu'avec des nombres.

```
function division(dividend: number, divisor: number) {  
    return dividend / divisor;  
}
```

```
division(10, 5);  
division("test", 50);
```

Argument of type 'string' is not assignable to parameter of type 'number'.



Commençons par un exemple simple : une fonction de division

Deuxième contrainte : il est impossible de diviser par `0`. Et c'est là que les problèmes commencent...

Certains proposeraient de résoudre ce problème de cette manière :

```
function division(dividend: number, divisor: number) {  
  if(divisor === 0) {  
    return null;  
  }  
  
  return dividend / divisor;  
}  
  
const result = division(8, 0);  
  
const result: number | null
```

Sauf qu'en mathématiques, une division ne renvoie jamais `null`. En faisant ça, il y a donc une incohérence avec le métier.



Commençons par un exemple simple : une fonction de division

Pour résoudre ce problème, je vous propose de commencer à jouer un peu avec TypeScript :

```
function division(dividend: number, divisor: number & { notZero: unknown }) {  
    return dividend / divisor;  
}
```

```
division(8, 0);
```

```
Argument of type 'number' is not assignable to parameter of type 'number & { notZero: unknown; }'.  
Type 'number' is not assignable to type '{ notZero: unknown; }'.
```

```
division(8, 2);
```

```
Argument of type 'number' is not assignable to parameter of type 'number & { notZero: unknown; }'.  
Type 'number' is not assignable to type '{ notZero: unknown; }'.
```

Ici, on a indiqué que l'argument `divisor` était un `number`, mais également un `objet` possédant une clé nommée `notZero`. Cette **intersection** de types nous empêche de mettre `0`, mais pour l'instant, elle nous empêche aussi de mettre n'importe quel autre `number`.



Commençons par un exemple simple : une fonction de division

Il nous faut donc être capable de fournir le type `number & { notZero: unknown }`. Pour cela, on va devoir créer une nouvelle fonction :

```
function numberIsNotZero(value: number) {  
  if (value === 0) return null;  
  return value as number & { notZero: unknown };  
}
```

```
const divisor = numberIsNotZero(12);
```

```
const divisor: (number & {  
  notZero: unknown;  
}) | null
```

Dans le cas où la valeur utilisée est égale à `0`, cette fonction nous renverra `null`. Sinon, elle renverra le type dont nous avons besoin.



Commençons par un exemple simple : une fonction de division

Il ne reste plus qu'à utiliser tout ça.

```
const divisor = numberIsNotZero(Math.random());  
if (divisor !== null) {  
  division(8, divisor);  
}
```

```
division(8, Math.random());
```

```
Argument of type 'number' is not assignable to parameter of type 'number & { notZero: unknown; }'.  
Type 'number' is not assignable to type '{ notZero: unknown; }'.
```

Il est donc impossible d'utiliser la fonction `division` sans appeler au préalable la fonction `numberIsNotZero`.



Quel est l'avantage de faire ça ?

- La déclaration des fonctions est tout de suite plus explicite et compréhensible. Le **diviseur** est explicitement autre chose que `0`.
- L'exécution est désormais sécurisée grâce au **typage**. Impossible d'utiliser `0`, donc impossible d'obtenir un résultat erroné à l'exécution.
- Le code de vérification est factorisé. Cette contrainte pourrait être utilisée par d'autres fonctions, ce qui éviterait d'avoir une condition répétée et un retour à `null` sur chacune d'entre elles à cause d'un paramètre qui serait égal à zéro.
- Moins de tests unitaires. Chaque fonction qui implémenterait cette vérification devrait être accompagnée de tests unitaires validant que les paramètres fournis ne créent pas de résultats problématiques. Il serait pourtant beaucoup plus pertinent de se concentrer sur des tests qui s'occupent uniquement du métier, et non de problèmes liés à des limitations techniques.
- Ce genre de contrat explicite permet de déléguer plus sereinement des tâches à des agents IA pour la partie d'implémentation.



Après la théorie, la pratique avec DuploJS !

DuploJS a tout un arsenal d'outils pour qualifier facilement les contraintes.

À votre disposition, vous avez :

- Les `DataParsers`
- Les `Primitives`
- Les `Constraints`
- Les `NewTypes`



DuploJS : Les DataParsers

Les **DataParsers** sont des outils qui permettent de représenter le **type** et les **spécificités** d'une donnée au runtime. Ils vont servir de **guards**, car même si le **type** est correct, il faut bien que les données entrantes soient **validées** par quelque chose.

```
import { DPE, E } from "@duplojs/utils";

DPE.string().min(12);

const dataParser = DPE.object({
  prop1: DPE.string().regex(/^@/),
  prop2: DPE.number().max(20),
})

const result = dataParser.parse(null);

if(E.isRight(result)) {
  result;
}

const result: E.Success<{
  prop1: string;
  prop2: number;
}>
```



DuploJS : Les DataParsers

Les `DataParsers` sont accompagnés de `Checkers` pour définir des spécificités à la donnée.

Exemple : sur une `string`, on souhaiterait qu'elle ait un minimum de `5` caractères. Le `DataParser` va vérifier le `type` et le `Checker` va vérifier la longueur de la `string`.

```
import { DP, DPE, E } from "@duplojs/utils";

DPE.string().min(5);

const userNameDataParser = DP.string()
  .addChecker(
    DP.checkerStringMin(5),
  );

const errorResult = userNameDataParser.parse("miko");
const successResult = userNameDataParser.parse("mathcovax");
```



DuploJS : Les DataParsers

Il existe au total plus de 20 `DataParsers`. Ils vous aideront à définir correctement une **donnée** et/ou à la **transformer**. Il est possible de récupérer le **type** inféré de vos `DataParsers` à travers les interfaces `Input` et `Output`. Ici, la valeur de `prop2` se retrouve stringifiée en sortie.

```
import { DPE, S } from "@duplojs/utils";
```

```
const dataParser = DPE.object({  
  prop1: DPE.string(),  
  prop2: DPE.number().transform(S.to),  
});
```

```
type Input = DPE.Input<typeof dataParser>;
```

```
type Input = {  
  prop1: string;  
  prop2: number;  
}
```

```
type Output = DPE.Output<typeof dataParser>;
```

```
type Output = {  
  prop1: string;  
  prop2: `${number}`;  
}
```



DuploJS : Les Primitives

Les **Primitives** sont un élément central. Elles créent une structure qui enveloppe une valeur dans un objet, ce qui permet de cumuler des attributs comme des **Constraints**. Les primitives peuvent seulement envelopper les types : **string**, **number**, **bigint**, **boolean**, **TheDate** et **TheTime**.

```
import { C } from "@duplojs/utils";

const maybeAge = C.Number.create(12);
const Age = C.Number.createOrThrow(12);
```

Age. |

La valeur définie est **inaccessible**. Pour la **manipuler**, il est donc **obligatoire** d'utiliser les **opérateurs** qui peuvent être fournis par la librairie ou par vous-même.

```
import { C } from "@duplojs/utils";

const result = C.subtract(
  C.Number.createOrThrow(12),
  C.Number.createOrThrow(10),
);
```



DuploJS : Les Constraints

Les **Constraints** permettent de vérifier qu'une valeur respecte une condition qui ne peut être vérifiée qu'au runtime. Pour créer une **Constraint**, il suffit d'indiquer un nom, la **Primitive** concernée et un ou plusieurs **checkers**.

```
import { C, DP } from "@duplojs/utils";

const PhoneNumber = C.createConstraint(
  "phone-number",
  C.String,
  DP.checkerRegex(/^+33[1-9]\d{8}$/)
);
type PhoneNumber = C.GetConstraint<typeof PhoneNumber>;

const phoneNumber = PhoneNumber.createOrThrow("+33612345678");

const phoneNumber: C.ConstrainedType<"phone-number", "+33612345678">
```

Pour créer le **contrat**, il suffit de déclarer un **type** et d'appeler l'interface **GetConstraint**.



DuploJS : Les Constraints

Pour ensuite exiger la contrainte, il suffit d'appeler le type défini.

```
declare function sendMessage(  
  params: {  
    phoneNumber: PhoneNumber,  
    content: string  
  }  
): Promise<void>  
  
sendMessage({  
  phoneNumber: PhoneNumber.createOrThrow("+33612345678"),  
  content: "this is super message",  
})
```

Il est donc maintenant impossible d'appeler notre fonction avec autre chose qu'un numéro de téléphone, ce qui nous garantit, avant même l'exécution, que tout se passera correctement.



DuploJS : Les Constraints

Mais dans la réalité, nous sommes souvent dépendants de plusieurs contraintes. C'est là que **DuploJS** amène les `ConstraintSet`.

```
import { C } from "@duplojs/utils";

const Content = C.createConstraintsSet(
  C.String,
  [
    C.StringMin(10),
    C.StringMax(150),
  ],
);

const content = Content.createOrThrow("this is super content");

const content: C.Primitive<"this is super content"> & C.ConstrainedType<"string-min-10", string> & C.ConstrainedType<"string-max-150", string>
```

Pour créer notre `ConstraintSet`, ici nous utilisons des contraintes fournies par la lib. On peut observer que la valeur obtenue est une primitive en combinaison avec toutes les contraintes définies dans le set.



DuploJS : Les Constraints

Et à l'usage, il est encore tout à fait possible de demander autant de contraintes qu'on le souhaite.

```
declare function sendMessage(  
  params: {  
    phoneNumber: PhoneNumber,  
    content: C.StringMin<10> & C.StringMax<150>  
  }  
): Promise<void>  
  
sendMessage({  
  phoneNumber: PhoneNumber.createOrThrow("+33612345678"),  
  content: Content.createOrThrow("this is super message"),  
})
```



DuploJS : Les Constraints

Les types constraints sont aussi capables d'être utilisés avec les opérateurs.

```
import { C, DP } from "@duplojs/utils";

const result = C.subtract(
  C.Positive.createOrThrow(12),
  C.Number.createOrThrow(10),
);

result;

const result: C.Primitive<number>
```

Et en toute logique, le type perdra les contraintes qu'il avait, car après opération, il n'est plus sûr de les respecter.



DuploJS : Les Constraints

Il est aussi tout à fait possible de caster des contraintes dans le cas où, logiquement, une contrainte implique l'autre. La fonction `castConstraint` permet, à travers un calcul de typage, de pouvoir réassigner une contrainte.

```
declare function sendMessage(  
  params: {  
    phoneNumber: PhoneNumber,  
    content: C.StringMax<150>  
  }  
): Promise<void>  
  
const content = C.StringMax(100).createOrThrow("ThisIsSuperMessage");  
  
sendMessage({  
  phoneNumber: PhoneNumber.createOrThrow("+33612345678"),  
  content: C.castConstraint(content, C.StringMax(150)),  
});
```

Dans le cas inverse, si une contrainte n'inclut pas l'autre, une erreur de typage sera indiquée.



DuploJS : Les NewTypes

Les **NewTypes** viennent donner une identité à la donnée. Pour en créer, il suffit d'utiliser soit une **Primitive**, soit un **DataParser** pour des structures composées.

```
import { C, DPE } from "@duplojs/utils";

const userEmail = C.createNewType("UserEmail", C.String, C.Email);
type userEmail = C.GetNewType<typeof userEmail>;

const userName = C.createNewType("UserName", C.String, [C.StringMin(5), C.StringMax(35)]);
type userName = C.GetNewType<typeof userName>;

const Subscription = C.createNewType(
  "Subscription",
  DPE.object({
    from: DPE.date(),
    to: DPE.date(),
  })
);
type Subscription = C.GetNewType<typeof Subscription>
```

Il est également possible de fournir des **Constraints** afin de rendre notre **NewType** éligible à ces contraintes.



DuploJS : Les NewTypes

Il s'utilise de la même manière que les Constraints et les Primitives .

```
declare function sendEmail(  
  params: {  
    email: C.Email,  
    content: C.StringMin<10> & C.StringMax<150>  
  }  
): Promise<void>  
  
sendEmail({  
  email: UserEmail.createOrThrow("duplojs@gmail.com"),  
  content: Content.createOrThrow("this is a super message"),  
});  
  
const result = C.concat(C.String.createOrThrow("test-"), UserEmail.createOrThrow("duplojs@gmail.com"));
```

Il est également possible de manipuler les NewTypes avec des Constraints ou des Primitives pour réobtenir une nouvelle Primitive .



Ça donne quoi dans une situation réelle ?

Pour illustrer le flux de travail que vous pouvez avoir avec ces éléments, je vous propose de réaliser une création de compte sur une application.

Lorsqu'un utilisateur veut créer son compte sur mon application, il est obligé de fournir :

- une adresse email
- un nom
- un âge.

Contraintes :

- L'adresse email ne doit pas déjà être utilisée.
- Le nom doit uniquement contenir des lettres et doit contenir entre 5 et 35 caractères.
- L'âge doit être supérieur ou égal à 13 ans.



Création d'un utilisateur

La première chose à faire est de déclarer nos champs avec toutes leurs contraintes.

```
const UserEmail = C.createNewType("UserEmail", C.String, C.Email);
type UserEmail = C.GetNewType<typeof UserEmail>;

const OnlyLetter = C.createConstraint(
  "only-letter",
  C.String,
  DP.checkerRegex(/^[\A-Za-zÀ-Öø-öø-ÿ]*$/))
)
type OnlyLetter = C.GetConstraint<typeof OnlyLetter>;

const UserName = C.createNewType("UserName", C.String, [
  C.StringMin(5),
  C.StringMax(35),
  OnlyLetter
]);
type UserName = C.GetNewType<typeof UserName>;

const UserAge = C.createNewType("UserAge", C.Number, C.NumberMin(13));
type UserAge = C.GetNewType<typeof UserAge>;
```



Création d'un utilisateur

Ensuite, on vient créer notre fonction qui s'assure que l'email est correctement disponible.

```
declare function userEmailIsAvailable(userEmail: UserEmail): Promise<  
  | (UserEmail & C.Evidence<"available">)  
  | (UserEmail & C.Evidence<"already-use">)  
>;
```

L'implémentation de celle-ci n'a pas beaucoup d'importance. Ici, on illustre juste les contrats d'entrée et de sortie.

On peut observer ici un nouveau type qui se nomme `Evidence`. Il sert de preuve de passage à un endroit. Il peut être ajouté à un `NewType`, une `Primitive` ou une `Constraint`.



Création d'un utilisateur

Après, on crée notre fonction qui crée l'utilisateur.

```
declare function createUser(params: {  
  email: C.GetEvidenceResult<typeof userEmailIsAvailable, "available">;  
  name: UserName,  
  age: UserAge,  
}): Promise<void>;
```

Encore une fois, ici, l'intérêt de l'exemple ne réside pas dans l'implémentation, mais dans le contrat qui oblige à faire certaines choses. On peut voir ici que j'utilise l'interface `GetEvidentResult`, puis que je cite la fonction `userEmailIsAvailable` et `available`, la preuve exigée.



Création d'un utilisateur

Pour aller plus loin dans l'implémentation, on va utiliser la librairie `@duplojs/http`, qui permet de créer des serveurs HTTP typés. Elle met à disposition la méthode `toExtractParser`, qui crée un `DataParser` capable de transformer une valeur simple en `NewType` avec ses contraintes.

```
useRouteBuilder("POST", "/users")
  .extract({
    body: {
      email: UserEmail.toExtractParser(),
      name: UserName.toExtractParser(),
      age: UserAge.toExtractParser(),
    }
  })
  .handler(
    [ResponseContract.conflict("user.email.alreadyUse"), ResponseContract.created("user.created")],
    async({ email, ... rest }, { response }) => {
      const emailIsAvailable = await userEmailIsAvailable(email);
      if(C.hasEvidence(emailIsAvailable, "already-use")) {
        return response("user.email.alreadyUse")
      }
      await createUser({ email: emailIsAvailable, ... rest });
      return response("user.created")
    }
  );
```



Pour conclure

Nous pouvons observer que grâce au **Type Driven Design** et aux outils de **DuploJS**, il est tout à fait possible de typer tous les aspects de nos données. Cela permet de définir précisément ce que nous souhaitons et de sécuriser l'exécution du code. Cela crée en plus une source de vérité unique, qui peut être utilisée à tous les niveaux, que ce soit pour valider en backend ou même en frontend dans des formulaires.



Fin du chapitre de l'intypable.

Prochain chapitre : Les entités.

